Föreläsning 10

Heap, heapsort, icke jämförande sortering

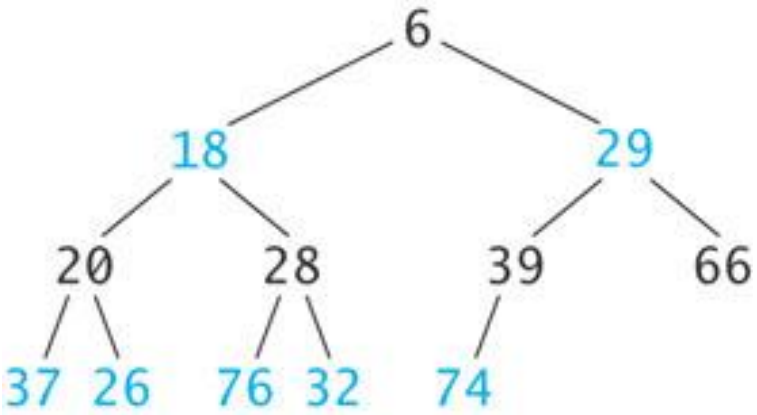
Föreläsning 10

- Heap
- Implementering av Heap
- Heapsort
- Counting sort
- Radix sort

Heap

En heap är ett komplett binärt träd där:

- rotens nyckel är en av de minsta nycklarna i trädet
- varje icke tomt underträd är en heap

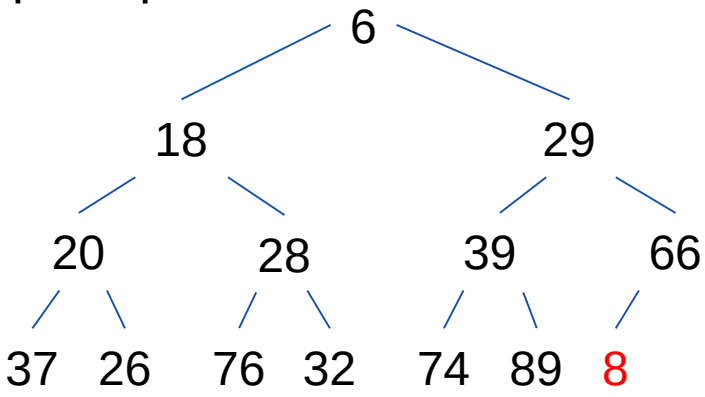


Exemplet ovan kallas ibland en min-heap. I en max-heap är istället rotens nyckel störst.

Insättning - O(log *n*)

Algoritm:

1. Sätt in nytt element på första lediga plats
2. Medans elementet inte är i rotnoden och elementets nyckel är mindre än förälderns nyckel:
   byt plats på förälder och elementet



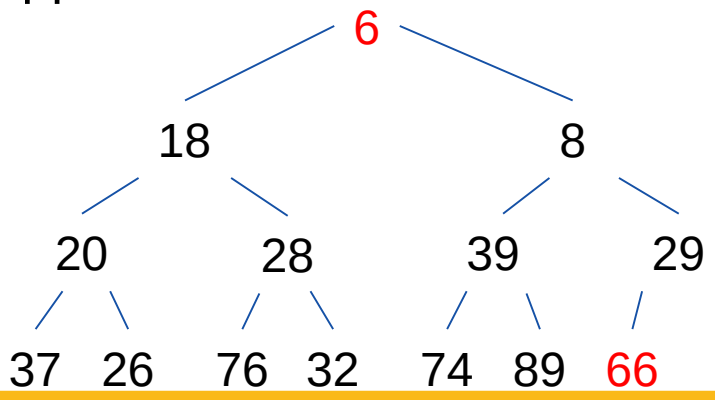
Borttagning - O(log *n*)

Vi tar endast bort elementet i rotnoden

Algoritm

1. Ersätt elementet i rotnoden med elementet i sista noden i trädet (kalla elementet S)
2. Så länge S har barn och S har större nyckel än något av sina barn:

   swappa S med minsta barnet



Detta resulterar i att vi när vi plockar ut element alltid får det med minst nyckel!

Användningsområden

- Prioritetskö

  Eftersom vi alltid får ut elementet med minst nyckel oberoende ordning vi stoppar in saker fungerar datastrukturen perfekt som en prioritetskö.

- Heap-sort

  Om vi stoppar in alla element vi vill sortera i en heap och sedan tar ut alla element kommer de att komma ut sorterade.

Heap i Java

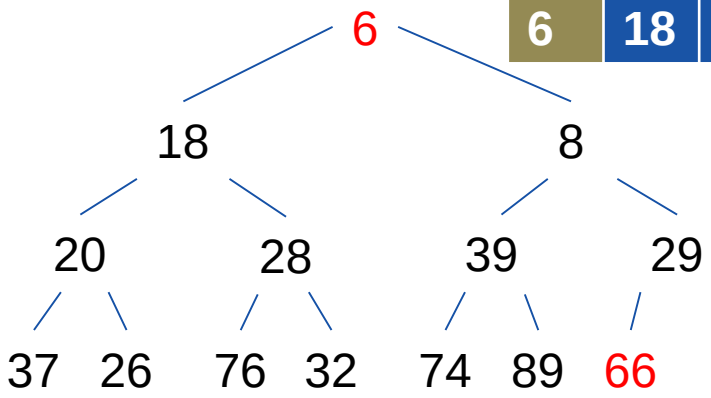
Java tillhandahåller en heap: java.util.PriorityQueue<E>

Precis som vid sortering måste objekt antingen implementera Comparable eller så måste en Comparator skickas med konstruktorn.

Implementering av en heap

Eftersom en heap är ett komplett binärt träd är det onödigt att implementera det som en länkad datastruktur. En array fungerar perfekt. Vi lägger helt enkelt nivåerna med elementen i ordning från vänster till höger. En nod med index p har då vänster barn med index 2p+1, höger barn med index 2p+2 och sin förälder med index (p-1)/2.

6	18	8	20	28	39	29	37	26	76	32	74	89	66



Heapsort

Vi ska sortera en array med *n* stycken element.

Algoritm:

Sätt in elementen ett efter ett i en heap

Ta ut elementen ett efter ett och sätt tillbaka dem i arrayen

Problem: Kräver *n* extra minne. Detta visar sig onödigt då vi kan implementera heapsort utan att använda extra minne. För att kunna använda arrayen som minne för heapen ska vi använda en max-heap där rotens nyckel är största nyckeln i trädet.

Heapsort in-place

Heapen kommer att ha rotnoden på plats 0 i arrayen och vara lagrad såsom vi beskrev ovan. Vi använder en max-heap. Heapens storlek kallar vi m. Vi sorterar en array a.

- offer(a, m) – sätter in elementet a[m] på plats m i arrayen i heapen som alltså består av elementen 0 till m-1
- poll(a,m) – tar bort ett element ur heapen och returnerar detta. m är heapens storlek.

```
m = 1
Medans m<n
        offer(a,m++)
Medans m>1
        a[m-1]=poll(a,m)
        m--
```

Läs mer i boken!

Heapsort in place

Input: |4 7 2 1 3
Remove from sequence and place in heap:
heap **|** inS
4|7 2 1 3
7 4|2 1 3
7 4 2|1 3
7 4 2 1|3
7 4 2 1 3| Remove min from heap and place in output sequence:
heap **|** outS
4 3 2 1|7
3 1 2|4 7
2 1|3 4 7
1|2 3 4 7
Output: 1 2 3 4 7|

Analys

För både jämförelser och tilldelningar gäller:

- För varje element n måste vi sätta in det i heapen: O(*n*log *n*)
- För varje element n måste vi ta ut det ur heapen: O(*n*log *n*)
- Så komplexiteten blir O(*n*log *n*)

Det kräver inget extra minne, minneskrav: O(*1*)

Stabilitet: Nej, den går inte att få stabil.

Användning:

Den är i praktiken normalt långsammare än Quick-sort men har fördelen att den har worst-case O(*n*log *n*). Quick-sort har worst- case O(n^2). Detta gör att den föredras i inbyggda system och säkerhetssystem (linux kärna). Jämfört med mergesort gäller att merge-sort använder cash effektivare och oftast är snabbare, parallelliserar bättre och är stabil men merge-sort kräver extra utrymme.

Heapify

Det går att effektivisera insättningen om man använder ett annat sätt att bygga upp trädet. Uppbyggandet av trädet får då komplexiteten O(n) istället för O(nlogn). Denna metod som brukar kallas heapify är något svårare att förstå än den vi använt och är också svårare att analysera. Vi ska i denna kurs nöja oss med den enklare algoritmen beskriven tidigare.

Implementering av en heap

Vi ska välja att implementera en generic heap där objekten som stoppas in ska vara Comparable, alternativt kan man sortera efter en nyckel (<K,V>) som vi gjorde för hastabellen. Ni får här skalet men får skriva metoderna själva. Enklast är att välja en rekursiv lösning.

```
public class Heap<E extends Comparable<E>> {

    private int size;
    private E[] data;
    private int nextPos;

    public Heap(int size) {
        this.size = size;
        data = (E[]) new Comparable[size];
        nextPos = 0;
    }

    public boolean insert(E data) {

    }

    public E extract() {

    }

}
```

Counting sort

Ide: vi räknar hur många vi har av varje möjligt värde

Om vi ska sortera en array av positiva heltal kan detta göras mycket enkelt:

```
private static void countingSortInt(int[] v) {
    int[] count = new int[findMax(v)+1];
    for(int a:v) count[a]++; //count[i] blir antalet gånger talet i förekommer
    int i=0;
    for(int j=0;j<count.length;j++)
        while(count[j]-->0) v[i++]=j;
}
```

Tilldelningar blir O(*n*+*k*), där k största heltalet

(ej jämförande algoritm och kan därför vara bättre än O(*n*log*n*))

Minne: O(k) extra minne vilket betyder att om största heltalet är stort blir den minneskrävande.

Alltså effektiv om största heltalet är litet.

Hitta max kan göras med:
Arrays.*stream*(v).parallel().max().getAsInt()

Counting sort

Om vi inte ska sortera heltal utan faktiska objekt (dock med nycklar som är positiva heltal) i en array behöver vi flytta elementen. Vi behöver då en extra array. (Exemplet använder heltal för enkelhetens skull)

```
private static void countingSort(int[] v) {
    int[] count = new int[findMax(v)+1], output = new int[v.length];
    for(int a:v) count[a]++;
    //count[i] är nu antalet gånger talet i förekommer
    int total=0,oldCount;
    for(int i=0;i<count.length;i++){
        oldCount=count[i];
        count[i]=total;
        total+=oldCount;
    }
    //count[i] är nu antalet tal mindre än i
    for(int a:v){//går igenom v och placerar varje tal på rätt plats mha count
        output[count[a]++]=a;
    }
    System.arraycopy(v, 0, output, 0, v.length);
}
```

Counting sort – exempel

index:	*0*	*1*	*2*	*3*	*4*	*5*
v	2	1	3	3	1	0
count	1	2	1	2		
count	0	1	3	4		
output				**2**		
count	0	1	**4**	4		
output		**1**		2		
count	0	**2**	2	4		
output		1		2	**3**	
count	0	2	2	**5**		
output		1		2	3	**3**
count	0	2	2	**6**		
output		1	**1**	2	3	3
count	0	**3**	2	6		
output	**0**	1	1	2	3	3
count	**1**	3	2	6		

```
for(int a:v) count[a]++;
```

```
int total=0,oldCount;
for(int i=0;i<count.length;i++){
    oldCount=count[i];
    count[i]=total;
    total+=oldCount;
}
```

```
for(int a:v) output[count[a]++]=a;
```

Bucket sort

En till counting sort närbesläktad sorteringsalgoritm är bucket sort. Varje bucket innehåller då ett intervall av element, tex 0-9, 10-19, 20-29, …

1. Lopa igenom arrayen och placera varje element i rätt bucket
2. Sortera varje bucket (med annan metod eller med bucket sort)
3. Gå igenom varje bucket i ordning och placera elementen i original-arrayen

Detta kan beroende på implementation vara en jämförande algoritm.

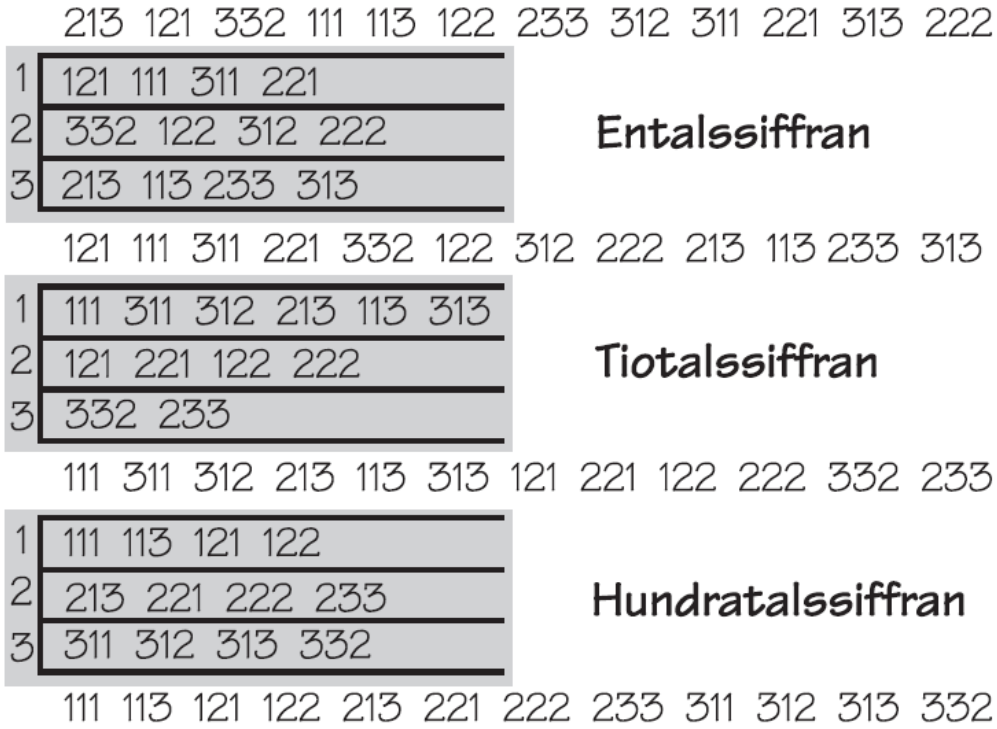
Radixsort

Kan ses som närbesläktad med bucket sort. Istället för att låta varje bucket motsvara ett intervall så motsvarar varje bucket en av siffrorna i talbasen (ex 0-9).

LeastSignificantDigit Radix sort:

- Gruppera talen baserat på den minst signifikanta siffran.
- Upprepa grupperingen för varje mer signifikant siffra.

Princip (för siffror 1-3 och tresiffriga tal):



Radixsort med counting sort

Ett effektivt sätt att implementera radixsort är att använda counting sort. En bra illustration hittar ni här:

https://www.cs.usfca.edu/~galles/visualization/RadixSort.html

Vi går först igenom materialet och räknar antalet tal med respektive slutsiffra.

Därmed vet vi var vi ska placera talen med respektive slutsiffra och kan sätta in dem på rätt plats.

Vi behöver då bara en extra array minne och minskar antalet gånger vi behöver flytta talen.